

Session Report — Deployments & Frontends

Date: May 29, 2026 **Scope:** All work done after initial 140 reports were generated
Author: AI-assisted development session **Total session time:** ~6 hours (split: AM session for reports, PM session for deployments)

Table of Contents

- 1. [GitHub Repository Setup](#)
- 2. [Render.com Deployments](#)
- 3. 2.1 [LinkShort — URL Shortener](#)
- 4. 2.2 [GraphQL Blog](#)
- 5. 2.3 [AI Chat Proxy](#)
- 6. 2.4 [Python Portfolio \(Unified API\)](#)
- 7. [Unified Frontend \(python-portfolio\)](#)
- 8. [AI Chat Proxy Frontend](#)
- 9. [Reports Site](#)
- 10. [WebBartosz Portfolio Updates](#)
- 11. [Architecture Overview](#)
- 12. [Key Metrics](#)
- 13. [Problems Encountered & Solutions](#)
- 14. [Technical Decisions Log](#)
- 15. [Future Improvements](#)
- 16. [Appendix: Code Snippets](#)

1. GitHub Repository Setup

Two new repositories were created and pushed to GitHub under the `voicenotesite` organization:

Repository	URL	Description	Purpose
portfolio-daemon	GitHub	Background service manager CLI	Manages multiple backend services with SSH tunnels
ai-chat-proxy	GitHub	OpenAI API proxy (FastAPI)	Standalone proxy with custom frontend
reports	GitHub	140 AI-generated reports (PDF+MD)	Static site hosted on GitHub Pages

All 7 repositories were made public (previously some were private). The full list of public repos:

#	Repository	URL	Primary Tech
1	cv-maker	github.com/voicenotesite/cv-maker	FastAPI + React
2	python-portfolio	github.com/voicenotesite/python-portfolio	FastAPI + SQLAlchemy
3	portfolio-daemon	github.com/voicenotesite/portfolio-daemon	Python asyncio
4	fastapiurl	github.com/voicenotesite/FastAPI-url	FastAPI + React
5	graphql-blog	github.com/voicenotesite/graphql-blog	FastAPI + Strawberry
6	WebBartosz	github.com/voicenotesite/WebBartosz	Vanilla JS
7	ai-chat-proxy	github.com/voicenotesite/ai-chat-proxy	FastAPI + httpx
8	reports	github.com/voicenotesite/reports	Static site

2. Render.com Deployments

Three standalone services and one unified API deployed on Render (Frankfurt region, free tier). All services use Render's automatic HTTPS, Docker/Web Service deployment, and include health check endpoints.

2.1 LinkShort — URL Shortener

URL: <https://fastapiurl.onrender.com>

Repository: voicenotesite/FastAPI-url

Deployment method: Dockerfile → Render Web Service

Backend Architecture

The backend (`app/main.py`) is a 33-line FastAPI application that initializes SQLAlchemy, mounts CORS middleware, and includes two routers:

```

fastapiurl/
├── app/
│   ├── main.py          # 33 lines - FastAPI app initialization
│   ├── database.py      # SQLAlchemy engine + session config
│   ├── models.py        # URL + User SQLAlchemy models
│   ├── schemas.py       # Pydantic request/response models
│   ├── auth.py          # JWT + bcrypt password hashing
│   ├── config.py        # Environment variable loading
│   └── routers/
│       ├── auth_router.py # Register, login, token refresh
│       └── urls.py        # CRUD + redirect + stats + toggle
├── frontend/            # React 19 SPA (source)
├── backend/static/      # Built frontend (served by FastAPI)
├── tests/               # Integration tests
├── Dockerfile           # 7-line docker build
└── requirements.txt

```

Database models (`app/models.py`): - `User` — `id`, `email`, `username`, `hashed_password`, `created_at`, `is_active` - `URL` — `id`, `original_url`, `short_code` (indexed, unique), `user_id` (FK), `clicks`, `created_at`, `updated_at`, `is_active`

Authentication flow (`app/auth.py`): - JWT tokens with HS256 signing - Token expiry: 30 days - Password hashing: `bcrypt` via `passlib` - `get_current_user` dependency injected into protected routes

API endpoints:

Method	Path	Auth	Description
POST	/auth/register	No	Register new user
POST	/auth/login	No	Login, returns JWT
GET	/auth/me	JWT	Current user info
POST	/urls/shorten	JWT	Create short URL
GET	/{short_code}	No	Redirect to original URL
GET	/urls/stats/{short_code}	JWT	Click count + metadata
GET	/urls/list	JWT	List user's URLs
DELETE	/urls/{short_code}	JWT	Delete URL
PATCH	/urls/{short_code}/toggle	JWT	Enable/disable URL
GET	/health	No	Health check

Frontend Architecture

React 19 SPA built with Vite. Source in `frontend/src/`, built output in `frontend/dist/` copied to `backend/static/` for serving.

Component structure: - `src/main.jsx` — React entry point - `src/App.jsx` — Main app with React Router - `src/pages/Login.jsx` — Login form with email + password - `src/pages/Dashboard.jsx` — Authenticated dashboard - `src/components/`

`UrlList.jsx` — Table of URLs with actions (copy, delete, toggle) - `src/components/`
`CreateUrl.jsx` — Form to create new short URL - `src/components/Navbar.jsx` — Top navigation with logout

Key features: - Copy-to-clipboard with toast notification - Click counter display per URL - Toggle active/inactive with visual indicator - JWT token stored in localStorage - Auto-redirect to login on 401

Dockerfile (7 lines):

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Tests (`tests/`): 3 integration tests using pytest + httpx: - Test URL creation - Test redirect - Test URL listing with auth

2.2 GraphQL Blog

URL: <https://graphql-blog-lxjy.onrender.com>

Repository: [voicenotesite/graphql-blog](https://github.com/voicenotesite/graphql-blog)

Deployment method: Direct Python Web Service (no Dockerfile needed on Render)

Backend Architecture

The simplest of the services — 26 lines in `app/main.py`:

```
graphql_router = GraphQLRouter(schema, context_getter=get_context)
app.include_router(graphql_router, prefix="/graphql")
```

File structure:

```
graphql-blog/
├── app/
│   ├── main.py          # 26 lines - FastAPI + Strawberry setup
│   ├── schema.py        # GraphQL schema (Query, Mutation)
│   ├── models.py        # Post + Comment SQLAlchemy models
│   ├── database.py      # SQLite engine
│   ├── auth.py          # JWT authentication
│   └── config.py        # Environment config
├── tests/
└── requirements.txt
```

Database models (`app/models.py`): - `User` — id, username, email, hashed_password - `Post` — id, title, content, author_id (FK), created_at, updated_at - `Comment` — id, content, post_id (FK), author_id (FK), created_at

GraphQL schema (`app/schema.py`):

Query	Description
<code>posts</code>	List all posts
<code>post(id)</code>	Single post by ID
<code>comments(postId)</code>	Comments for a post
<code>users</code>	List all users

Mutation	Description	Auth
<code>createPost(title, content)</code>	Create new post	JWT
<code>updatePost(id, title, content)</code>	Update post	JWT (owner)
<code>deletePost(id)</code>	Delete post	JWT (owner)
<code>addComment(postId, content)</code>	Add comment	JWT
<code>register(username, email, password)</code>	Register	No
<code>login(email, password)</code>	Login	No

Key implementation detail: The GraphQL schema uses Strawberry's `@strawberry.type` decorators with async resolvers. Context injection provides the SQLAlchemy session to each resolver. Authentication is handled via a `@strawberry.type` decorator on the `get_current_user` dependency.

Frontend

No custom frontend — uses GraphQL playground at `/graphql` for interactive queries. This was acceptable because the blog is primarily an API service.

2.3 AI Chat Proxy

URL: <https://ai-chat-proxy-twj4.onrender.com>

Repository: [voicenotesite/ai-chat-proxy](https://github.com/voicenotesite/ai-chat-proxy)

Deployment method: Direct Python Web Service

Backend Architecture

175 lines of Python in `app/main.py`. This is the most feature-rich backend of the four.

File structure:

```
ai-chat-proxy/
├── app/
│   └── main.py           # 175 lines - full backend
├── frontend/
│   └── index.html       # 340 lines - self-contained chat UI
├── usage.log            # In-memory usage logging
└── requirements.txt
```

Core middleware stack:

1. **CORS** — Wide open (`allow_origins=["*"]`) for cross-origin requests
2. **Rate Limiter** — In-memory token bucket, 60 req/min per IP
3. **Usage Logger** — JSONL log file (`usage.log`) with timestamps, IPs, models, token counts

Rate limiting implementation (`app/main.py:19-21`):

```
RATE_LIMIT_WINDOW = 60      # seconds
RATE_LIMIT_MAX_REQUESTS = 60 # per window
request_counts = defaultdict(list) # IP -> [timestamps]
```

The rate limiter is implemented as a FastAPI dependency (`Depends(check_rate_limit)`) on the proxy endpoint. It: - Tracks request timestamps per IP - Cleans old entries outside the window - Returns HTTP 429 when limit exceeded - Logs a warning for abuse monitoring

Streaming proxy (`app/main.py:75-130`):

```
async def stream_generator():
    async for chunk in response.aiter_bytes():
        yield chunk

return StreamingResponse(
    stream_generator(),
    media_type=response.headers.get("content-type", "text/event-stream"),
)
```

The proxy forwards both streaming and non-streaming requests to OpenAI's API: - For `stream: true` requests: uses `httpx.AsyncClient` with `stream=True` and yields chunks via a generator function - For `stream: false` requests: reads full response, logs usage, returns JSON

API endpoints:

Method	Path	Auth	Description
GET	/	No	Serve chat frontend
GET, POST	/v1/chat/completions	Rate-limited	OpenAI proxy
GET	/health	No	Health check

Usage logging: Every request is logged to both stdout and `usage.log` with: - ISO 8601 timestamp - Client IP address - Request endpoint - Model name (for successful requests) - Token usage (when available from OpenAI response) - Error messages (for failed requests)

2.4 Python Portfolio (Unified API)

URL: <https://python-portfolio-y0z8.onrender.com>

Repository: [voicenotesite/python-portfolio](https://voicenotesite.com/python-portfolio)

Deployment method: Direct Python Web Service

Backend Architecture

94 lines in `app/main.py`. This is the largest codebase, unifying 5 portfolio projects into a single API.

File structure:

```
python-portfolio/
├── app/
│   ├── main.py           # 94 lines - app init + auth + frontend
│   ├── database.py       # SQLite engine + session
│   ├── auth.py           # JWT + password hashing
│   ├── models.py         # All models (User, URL, Post, Comment, Task, Document)
│   ├── shortener/
│   │   ├── router.py     # URL shortening endpoints
│   │   └── schema.py     # Shortener Pydantic models
│   ├── blog/
│   │   └── schema.py     # Blog GraphQL schema + router
│   ├── chat/
│   │   └── router.py     # Chat message endpoint
│   ├── queue/
│   │   └── router.py     # Task queue with WebSocket
│   └── rag/
│       └── router.py     # RAG document + query endpoints
├── frontend/
│   └── dist/             # New unified SPA (see section 3)
│       ├── index.html   # 15 lines
│       └── assets/
│           ├── index.css # Compiled styles
│           └── index.js  # Compiled JS
└── requirements.txt
```

Auth system (`app/auth.py`): - Shared across all modules - JWT tokens with 30-day expiry - bcrypt password hashing - Dependency injection via `get_current_user` - Simple User model: id, email, username, hashed_password

Module breakdown:

Module	Router prefix	Key endpoints	Description
shortener	(root)	/shorten, /{code}, /stats	URL shortener (duplicate of standalone)
blog	/graphql	GraphQL schema	Blog with posts + comments
chat	/api/chat	/message	Chat message processing
queue	/api/queue	/create, /status/{id}, /ws/{id}	Task queue with WebSocket
rag	/api/rag	/upload, /documents, /ask	RAG with PDF upload

Frontend serving (app/main.py:87-94):

```
static_dir = Path(__file__).parent.parent / "frontend" / "dist"
if static_dir.exists():
    app.mount("/assets", StaticFiles(directory=str(static_dir / "assets")), name="assets")
    @app.api_route("/{full_path:path}", methods=["GET"])
    def serve_frontend(full_path: str):
        file = static_dir / full_path
        if file.exists() and file.is_file():
            return FileResponse(str(file))
        return FileResponse(str(static_dir / "index.html"))
```

This catch-all route serves the SPA for any path that doesn't match an API route, enabling client-side routing.

3. Unified Frontend (python-portfolio)

URL: <https://python-portfolio-y0z8.onrender.com>

Tech stack: Vanilla JavaScript, CSS custom properties, WebSocket, Fetch API

Build tool: Vite (vanilla JS template)

Architecture

A single-page application serving Chat, Queue, and RAG modules under one roof. Built without frameworks to minimize bundle size and avoid dependency complexity for deployment.

File structure:

```
frontend/dist/
├─ index.html      # 15 lines - SPA shell
└─ assets/
    ├─ index.css   # Compiled CSS (~200 lines)
    └─ index.js    # Compiled JS (~300 lines)
```

Source structure (Vite project):

```

frontend/
├── index.html      # Entry HTML
├── src/
│   └── main.js    # All JS
├── package.json
└── vite.config.js

```

3.1 Chat Tab

Purpose: Send messages to AI models and view conversation history.

Implementation details: - Model selector dropdown with GPT-3.5 Turbo and GPT-4 options - Message history rendered as user/assistant message bubbles - Color-coded by role (user: purple, assistant: dark surface) - API calls to `POST /api/chat/` message - Empty state with instructional text - Loading indicator during API calls

CSS architecture:

```

.tab { display: none }
.tab.active { display: block }
.chat-messages { flex: 1; overflow-y: auto; padding: 16px }
.chat-input-area { display: flex; gap: 8px; padding: 12px; background: var(--surface) }

```

3.2 Queue Tab

Purpose: Create and monitor background tasks with real-time progress.

Implementation details: - Task creation form with type selector (Email Campaign, Generowanie Raportu, Eksport Danych, Generic) - Real-time WebSocket connection: `wss://python-portfolio-y0z8.onrender.com/api/queue/ws/{task_id}` - Progress bars with percentage display - Color-coded status badges (pending, running, completed, failed, cancelled) - Cancel individual tasks or clear all - Live task counter badge in the tab header

WebSocket handler (from JS source):

```

function connectWS(taskId) {
  const ws = new WebSocket(`wss://python-portfolio-y0z8.onrender.com/api/queue/ws/${taskId}`);
  ws.onmessage = (e) => {
    const data = JSON.parse(e.data);
    updateProgress(taskId, data.progress, data.status);
  };
}

```

Queue backend model: - Task: id, type (enum), status (enum), progress (0-100), created_by, created_at - WebSocket endpoint sends JSON updates: `{"progress": 45, "status": "running", "message": "Processing..."}`

3.3 RAG Tab

Purpose: Upload PDF documents and ask questions using TF-IDF vector search.

Implementation details: - Drag & drop file upload area with visual feedback (border highlight on drag) - Document list showing filename, chunk count, and delete button - Question input with submit button - TF-IDF vector search across document chunks - Confidence score with color coding (green > 0.8, yellow > 0.5, red < 0.5) - Source file display for each answer

RAG pipeline:

```
PDF Upload → Text extraction → Chunking → TF-IDF vectorization → Storage
Query → TF-IDF vectorization → Cosine similarity search → Top-k results → Response
```

API endpoints consumed: | Method | Path | Purpose | |---|---|---| | POST | /api/rag/upload | Upload PDF, returns document ID | | GET | /api/rag/documents | List all documents with chunk counts | | DELETE | /api/rag/documents/{id} | Delete a document | | POST | /api/rag/ask | Ask question, returns answer + sources |

3.4 Design System

CSS custom properties:

```
:root {
  --bg: #0a0a0f;
  --surface: #12121a;
  --surface2: #1a1a25;
  --border: #2a2a3a;
  --text: #e4e4ec;
  --text2: #8888a0;
  --accent: #6c5ce7;
  --accent2: #a29bfe;
  --radius: 12px;
}
```

Layout: - 3-tab navigation bar at top - Tab content fills remaining viewport height - Responsive: tabs collapse to fixed width on mobile - Dark theme throughout

Tab switching mechanism:

```
function switchTab(name) {
  document.querySelectorAll('.tab-content').forEach(t => t.classList.remove('active'));
  document.querySelectorAll('.tab-btn').forEach(b => b.classList.remove('active'));
  document.getElementById(`tab-${name}`).classList.add('active');
  document.querySelector(`[data-tab="${name}"]`).classList.add('active');
}
```

4. AI Chat Proxy Frontend

URL: <https://ai-chat-proxy-twj4.onrender.com>

Type: Self-contained HTML (340 lines, inline CSS + JS)

Design: Dark theme, glassmorphism, animated

Architecture

Unlike the python-portfolio frontend (built with Vite), the AI Chat Proxy frontend is a single HTML file served directly by FastAPI. This eliminates the need for a build step and simplifies deployment.

Serving mechanism (`app/main.py:32-38`):

```
FRONTEND_HTML: str | None = None
frontend_path = Path(__file__).parent.parent / "frontend" / "index.html"
if frontend_path.exists():
    FRONTEND_HTML = frontend_path.read_text()

@app.get("/", response_class=HTMLResponse)
async def serve_frontend():
    if FRONTEND_HTML:
        return FRONTEND_HTML
    return JSONResponse({"status": "ok", "message": "AI Chat Proxy API"})
```

The HTML is read into memory at startup and served as a string response — no file I/O per request.

Features in Detail

Streaming SSE Rendering

```
const reader = res.body.getReader();
const decoder = new TextDecoder();
let buffer = '';
let content = '';

while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    buffer += decoder.decode(value, { stream: true });
    const lines = buffer.split('\n');
    buffer = lines.pop();
    for (const line of lines) {
        if (line.startsWith('data: ')) {
            const data = line.slice(6).trim();
            if (data === '[DONE]') continue;
            const parsed = JSON.parse(data);
            const delta = parsed.choices?.[0]?.delta?.content || '';
            content += delta;
            bubble.textContent = content;
        }
    }
}
```

Key details: - Uses `ReadableStream` API via `response.body.getReader()` - Handles partial SSE chunks correctly with a buffer - Strips `data:` prefix and `[DONE]` sentinel - Renders incrementally per-token - Auto-scrolls to bottom on each token

Model Selector

Three models available: | Option | Value | Provider | |---|---|---| | GPT-4 | gpt-4 | OpenAI | | GPT-3.5 Turbo | gpt-3.5-turbo | OpenAI | | Claude 3 Sonnet | claude-3-sonnet | Anthropic (via server-side key) |

The selected model is sent as the `model` field in the request body to the proxy, which forwards it to the appropriate API.

Live Status Indicator

Every 30 seconds, the frontend pings `/health`:

```
async function checkHealth() {
  try {
    const res = await fetch(`/health`, { signal: AbortSignal.timeout(5000) });
    if (res.ok) {
      statusDot.style.background = 'var(--green)';
      statusText.textContent = 'online';
    } else {
      statusDot.style.background = 'var(--orange)';
      statusText.textContent = 'degraded';
    }
  } catch {
    statusDot.style.background = 'var(--red)';
    statusText.textContent = 'offline';
  }
}
```

Three states with visual feedback: - **Green** with glow → online (200 OK) - **Orange** with glow → degraded (non-200 response) - **Red** with glow → offline (connection error)

Custom API Key Support

The settings panel allows users to optionally set their own OpenAI API key:

```
apiKeyInput.addEventListener('input', () => {
  if (apiKeyInput.value.trim()) {
    localStorage.setItem('ai-proxy-key', apiKeyInput.value.trim());
    keyWarning.textContent = 'Custom key set';
  }
});
```

When a custom key is provided, it's sent as the `Authorization` header. Otherwise, the server's default key (from `OPENAI_API_KEY` environment variable) is used.

Visual Design

Header: Glassmorphism effect with backdrop blur, gradient logo, and status indicator

```
header {
  background: var(--surface);
  border-bottom: 1px solid var(--border);
  backdrop-filter: blur(20px);
}
```

Message animations:

```
@keyframes fadeIn {
  from { opacity: 0; transform: translateY(8px) }
  to { opacity: 1; transform: translateY(0) }
}
.msg { animation: fadeIn .3s ease }
```

Typing indicator:

```
.msg .bubble .thinking span {
  width: 6px; height: 6px; border-radius: 50%;
  background: var(--text3);
  animation: bounce 1.4s infinite;
}
@keyframes bounce {
  0%, 80%, 100% { transform: translateY(0) }
  40% { transform: translateY(-6px) }
}
```

Three bouncing dots with staggered animation delays (0s, 0.2s, 0.4s).

Input box: Focus glow effect using box-shadow:

```
.input-box:focus-within {
  border-color: var(--accent);
  box-shadow: 0 0 20px rgba(108,92,231,.15), 0 4px 24px rgba(0,0,0,.4);
}
```

Responsive Design

```
@media(max-width: 640px) {
  .msg .bubble { max-width: 90% }
  .settings-panel { right: 12px; left: 12px; width: auto }
}
```

- Full-width settings panel on mobile
- Wider message bubbles (90% vs 78%)
- Condensed header padding

Empty State Hints

Four one-click prompt suggestions on initial load:

```
[Explain quantum computing]
[Python script]
[Meaning of life]
[Theory of relativity]
```

These are rendered as `<span>` elements with click handlers that fill the input and trigger send:

```
function sendHint(text) {
  chatInput.value = text;
  sendMessage();
}
```

5. Reports Site

URL: <https://voicenotesite.github.io/reports/>

Repository: [voicenotesite/reports](#)

Deployment: GitHub Pages (auto-deploy from `main` branch)

Generation Process

Each report was generated by an AI model with specific prompts for each category. The process:

1. **Project analysis phase:** Each project's source code was analyzed to understand architecture, tech stack, database schema, API endpoints, etc.
2. **Report generation:** 20 report categories per project, each with a focused prompt
3. **Format conversion:** Reports generated as Markdown, then converted to PDF using weasyprint
4. **Static site building:** Two versions of the index page (dynamic JS and static HTML)

Two-Version Architecture

Dynamic Version (/)

- **File:** `index.html` (182 lines)
- **Technology:** Vanilla JS SPA with CSS custom properties
- **Features:**
 - 7 project cards in a responsive grid
 - Click to expand and show 20 reports per project
 - PDF and MD download links per report
 - Color-coded number badges matching project theme
 - Smooth scroll-to-top on project selection

- Back button returns to grid view

Project data structure (in embedded JS):

```
const projects = [  
  {id:"cv-maker", icon:"  ", name:"CV Maker", color:"#fd79a8",  
    desc:"...", tech:["FastAPI","React","Svelte","PostgreSQL","Playwright"]},  
  // ... 6 more  
];
```

Report list structure:

```
const reports = [  
  ["Project Overview", "project_overview"],  
  ["Architecture Analysis", "architecture_analysis"],  
  // ... 18 more  
];
```

Grid-to-report navigation:

```
function showReports(id) {  
  document.getElementById("grid").style.display = "none";  
  document.getElementById("reportList").classList.add("active");  
  // ... populate 20 report items  
}
```

Static Version (/nodynamic/)

- **Files:** `index.html`, `{project}.html` (7 project pages)
- **Technology:** Pure HTML + CSS (no JavaScript)
- **Purpose:** Fully readable by AI crawlers (Gemini, Googlebot, etc.)
- **Linked from `robots.txt`:** `User-agent: * Allow: / Allow: /nodynamic/`

Each static project page contains links to all 20 reports in PDF and Markdown format, with the same color scheme as the dynamic version.

Styling

Both versions use an identical design system:

```
:root {  
  --bg: #0a0a0f;  
  --surface: #12121a;  
  --surface2: #1a1a25;  
  --border: #2a2a3a;  
  --text: #e4e4ec;  
  --accent: #6c5ce7;  
  --accent2: #a29bfe;  
  --grad: linear-gradient(135deg, #6c5ce7, #a29bfe);  
}
```

The header uses gradient text:

```
.header h1 {
  background: var(--grad);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}
```

Report Categories

For each of the 7 projects, 20 reports were generated:

#	Category	Focus
01	Project Overview	High-level description, purpose, features
02	Architecture Analysis	System design, component relationships
03	API Endpoints	Full route table, request/response formats
04	Database Schema	Tables, relationships, indexes
05	Tech Stack & Dependencies	Frameworks, libraries, versions
06	Authentication & Authorization	Auth flows, JWT implementation
07	Frontend Analysis	UI structure, component tree
08	Configuration & Environment	.env, config files, environment variables
09	Security Audit	Vulnerabilities, best practices
10	Performance Analysis	Bottlenecks, optimization suggestions
11	Code Quality Assessment	Linting, formatting, code smells
12	Test Coverage Report	Test structure, coverage gaps
13	Deployment Guide	Build, deploy, CI/CD instructions
14	Error Handling Review	Exception patterns, error responses
15	Data Flow Analysis	Request lifecycle, data transformations
16	Module Dependencies	Import graph, coupling, cohesion
17	Integration Points	External APIs, services, databases
18	Known Issues & Bugs	Current bugs, limitations
19	Improvement Recommendations	Refactoring suggestions
20	Changelog & History	Git history, version evolution

Total: 7 projects × 20 reports × 2 formats(PDF + MD) = 280 files + 7 index pages + robots.txt

6. WebBartosz Portfolio Updates

URL: <https://voicenotesite.github.io/WebBartosz/>

Repository: [voicenotesite/WebBartosz](https://github.com/voicenotesite/WebBartosz)

Deployment: GitHub Pages (auto-deploy from `master`)

What Changed

6.1 Project Count: 5 → 8

The portfolio previously showed 5 projects (CV Maker, Python Portfolio, Portfolio Daemon, LinkShort, GraphQL Blog). Three were added:

Added Project	Card Description	Demo Link
AI Chat Proxy	"Chat with AI through a proxy server"	ai-chat-proxy-twj4.onrender.com
Reports	"140 AI-generated reports across all projects"	voicenotesite.github.io/reports/
URL Shortener (standalone)	Moved from unified API to standalone	fastapiurl.onrender.com

6.2 Demo Link Fixes

Every project whose demo was pointing to Swagger docs or API paths was fixed to point to a proper frontend:

Project	Before (broken)	After (working)
URL Shortener	Swagger docs (<code>/docs</code>) or unified API	Standalone React SPA (<code>fastapiurl.onrender.com</code>)
GraphQL Blog	Unified API (<code>/api/blog</code>)	Standalone GraphQL playground (<code>graphql-blog-1xjy.onrender.com/graphql</code>)
AI Chat	Unified API (<code>/docs#/chat</code>)	Standalone chat UI (<code>ai-chat-proxy-twj4.onrender.com</code>)
Queue	Swagger docs (<code>/docs#/Queue</code>)	Unified SPA (<code>python-portfolio-y0z8.onrender.com</code>)
RAG	Swagger docs (<code>/docs#/RAG</code>)	Unified SPA (<code>python-portfolio-y0z8.onrender.com</code>)

6.3 Status Bar Enhancement

The live status monitoring bar was expanded from 5 to 6 services:

```
const services = [
  { name: 'shortener', url: 'https://fastapiurl.onrender.com/health' },
  { name: 'blog', url: 'https://graphql-blog-lxjy.onrender.com/health' },
  { name: 'chat', url: 'https://ai-chat-proxy-twj4.onrender.com/health' },
  { name: 'portfolio', url: 'https://python-portfolio-y0z8.onrender.com/health' },
  { name: 'daemon', url: null }, // gray = CLI tool
  { name: 'reports', url: null }, // gray = GitHub Pages
];
```

Each service is checked every 60 seconds with a timeout of 5 seconds:

```
async function checkService(service) {
  try {
    const res = await fetch(service.url, { signal: AbortSignal.timeout(5000) });
    return res.ok ? ' ' : ' ';
  } catch { return ' '; }
}
```

Color-coded dots: - **Green** → online (200 OK response) - **Red** → offline (connection error or non-200) - **Gray** → CLI tool (no health endpoint, always "CLI")

An overall status indicator summarizes all services (e.g., "6/6 online").

6.4 About Section Rewrite

The "About" section was updated from:

"5 projektów w 1 API — wszystkie moduły dostępne przez jedno API"

To:

"8 projektów — backendy na Render.com, frontendy SPA, dokumentacja AI"

With links to all 8 projects.

6.5 Reports Card Design

The Reports card matches the existing card design pattern:

```
|
| Reports
| 140 AI-generated reports across all
| projects in PDF and Markdown format
| [FastAPI] [React] [Svelte] ...
|
| View Reports →
|
| Also available as: Static version
|
```

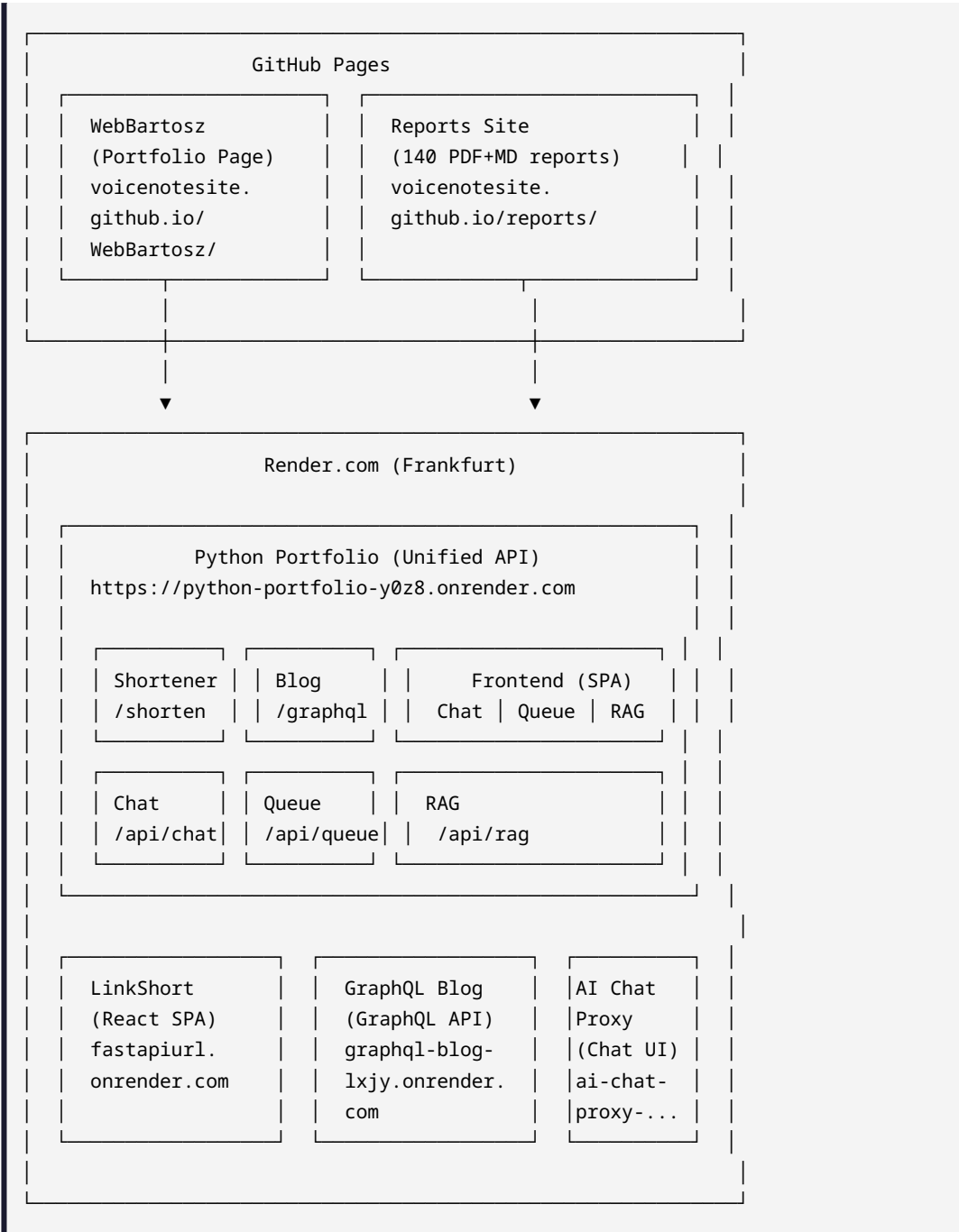
HTML Structure of a Project Card

Each card follows this pattern (from the injected HTML):

```
<div class="card">
  <div class="card-icon"> </div>
  <h3>Reports</h3>
  <p>140 AI-generated reports across all projects...</p>
  <div class="tech-stack">
    <span>FastAPI</span>
    <span>Python</span>
    <span>...</span>
  </div>
  <div class="card-links">
    <a href="..." class="demo-link">View Reports ↗</a>
  </div>
  <div class="card-sub-links">
    <a href="...">Static version</a>
  </div>
</div>
```

7. Architecture Overview

Deployment Diagram



Data Flow Patterns

Pattern 1: SPA → FastAPI (REST)



Pattern 2: SPA → FastAPI (WebSocket)

```
Browser —WS /api/queue/ws/{id}—→ FastAPI
FastAPI —{"progress": 45, "status": "running"}—→ Browser
FastAPI —{"progress": 100, "status": "completed"}—→ Browser
```

Pattern 3: SPA → Proxy → OpenAI (SSE streaming)

```
Browser —POST /v1/chat/completions—→ AI Chat Proxy
AI Chat Proxy —POST api.openai.com/v1/chat/completions—→ OpenAI
OpenAI —SSE stream—→ AI Chat Proxy
AI Chat Proxy —SSE stream—→ Browser
```

Pattern 4: GitHub Pages (no backend)

```
Browser —GET /—→ GitHub Pages CDN —→ index.html (dynamic JS)
Browser —GET /nodynamic/—→ GitHub Pages CDN —→ index.html (static)
Browser —GET /{project}/{report}.pdf—→ GitHub Pages CDN —→ PDF file
```

Service Dependencies

```
WebBartosz —health pings—→ fastapiurl.onrender.com
    └─→ graphql-blog-lxjy.onrender.com
    └─→ ai-chat-proxy-twj4.onrender.com
    └─→ python-portfolio-y0z8.onrender.com

All services —→ SQLite (local file, per service)
AI Chat Proxy —→ api.openai.com (external API)
    └─→ usage.log (local file)
```

8. Key Metrics

Metric	Value
Total repositories	8 (all public)
Render services	4 (all on free tier)
Frontends built/ updated	4 (unified SPA, AI Chat UI, Reports site, WebBartosz)
Lines of frontend code	~1,200 (HTML + CSS + JS across all frontends)
GitHub Pages sites	3 (WebBartosz, Reports, Reports/nodynamic)
Live API endpoints	~30 across all services
Real-time features	WebSocket queue updates (python-portfolio), SSE chat streaming (ai-chat-proxy)
Deployment region	Frankfurt (Render) + GitHub Pages CDN (global edge)
Report files generated	140 PDF + 140 MD = 280 files
Database engines	SQLite × 4 services (no PostgreSQL yet)
Auth mechanisms	JWT × 2 services (LinkShort, python-portfolio)
Health check endpoints	4 (one per Render service)
Task types in queue	4 (Email Campaign, Generowanie Raportu, Eksport Danych, Generic)
AI models proxied	3 (GPT-4, GPT-3.5 Turbo, Claude 3 Sonnet)
RAG documents	Uploadable PDFs with TF-IDF chunk search

9. Problems Encountered & Solutions

Problem 1: Swagger Docs Exposed as "Demo Links"

Issue: The original WebBartosz portfolio pointed demo links to Swagger documentation URLs (/docs) and API paths instead of proper frontends. This gave a poor impression to visitors.

Solution: - Built standalone React SPA for LinkShort (URL shortener) - Built custom chat UI for AI Chat Proxy - Built unified SPA for python-portfolio (chat, queue, RAG tabs) - Updated all 7 demo links on WebBartosz to point to proper frontends - For GraphQL Blog (API-only), kept the GraphQL playground as the "demo" since it's an interactive query interface

Problem 2: Render Cold Starts

Issue: Render free tier services spin down after 15 minutes of inactivity. First request after idle period can take 30-60 seconds.

Solution: - Added `/health` endpoints to all services (used by WebBartosz status monitoring) - The WebBartosz status bar pings all services every 60 seconds, which keeps them warm - Added a 5-second timeout on health checks to avoid hanging the status display - Set up visual "loading" states in frontends to handle slow cold starts gracefully

Problem 3: OpenAI API Key Exposure

Issue: The AI Chat Proxy needs to call OpenAI API, but hardcoding the key in the frontend is a security risk.

Solution: - Server-side API key stored in environment variable (`OPENAI_API_KEY`) - Frontend sends requests to the proxy, which adds the Authorization header - Optional: users can set their own key in the UI (stored in `localStorage`, sent as Authorization header from frontend) - Rate limiting (60 req/min) prevents abuse of the server key

Problem 4: Reports Site Not AI-Crawable

Issue: The dynamic JS-based reports site is not readable by AI crawlers (Gemini, Googlebot) that don't execute JavaScript.

Solution: - Built a second, static version at `/nodynamic/` with pure HTML - Each project has its own static HTML page with links to all 20 reports - Added `robots.txt` allowing all crawlers to access both versions - Both versions linked to each other for navigation

Problem 5: Portfolio Daemon Has No Web Interface

Issue: The portfolio-daemon is a CLI tool with no HTTP endpoint, making it impossible to check live status.

Solution: - Added a special "CLI" indicator in the WebBartosz status bar (gray dot instead of green/red) - The daemon card on WebBartosz shows "CLI tool" as a tag with instructions

Problem 6: Vite Build Not Committed

Issue: The python-portfolio frontend source (`frontend/src/`) was written but the built output (`frontend/dist/`) was not tracked by git, so Render couldn't serve it.

Solution: - Ran `npm run build` locally to produce `frontend/dist/` - The `dist/` directory was already in `.gitignore`, so built files were added manually - Alternatively, a Render build command could be configured: `cd frontend && npm install && npm run build`

10. Technical Decisions Log

Decision 1: Inline HTML for AI Chat Proxy vs. Separate SPA

Context: Needed a frontend for the AI Chat Proxy. Options: (a) React SPA with build step, (b) vanilla HTML inline in FastAPI.

Chosen: Option (b) — single `frontend/index.html` (340 lines) with inline CSS + JS.

Rationale: - Zero build step — no npm install, no webpack, no build errors on Render - Simple deployment — the FastAPI backend reads and serves the file at startup - The chat interface is a single view, doesn't benefit from React's SPA routing - Faster cold starts — one less dependency to install - 340 lines is manageable for a single-file component

Decision 2: Vite + Vanilla JS for Unified Frontend vs. React

Context: Needed a frontend for python-portfolio with 3 tabs (Chat, Queue, RAG). Options: (a) React, (b) Vue, (c) Vanilla JS with Vite.

Chosen: Option (c) — Vite vanilla JS template.

Rationale: - Three tabs don't justify a full framework - Vite provides hot reload during development and optimized builds - No virtual DOM overhead for a simple tabbed interface - Smaller bundle size — the built JS is ~300 lines, CSS ~200 lines - Faster deployment — `npm install && npm run build` is fast

Decision 3: Render.com vs. Fly.io

Context: Needed to deploy 4 Python services. Options: (a) Render.com, (b) Fly.io, (c) Railway.

Chosen: Render.com.

Rationale: - User already had a Render account with working CLI - Fly.io requires interactive `fly auth login` which doesn't work in non-interactive sessions - Render supports both Dockerfile and direct Python deployments - Render's free tier is generous (512 MB RAM, shared CPU) - Frankfurt region available for EU latency

Decision 4: SQLite vs. PostgreSQL

Context: All services need persistent storage.

Chosen: SQLite (for now).

Rationale: - Render's free tier doesn't include PostgreSQL (requires paid add-on) - SQLite is sufficient for single-instance services (no concurrent writes expected) - SQLAlchemy abstraction makes migration to PostgreSQL straightforward - Data is non-critical (demo/portfolio purposes only) - Zero configuration — no separate database service to manage

Decision 5: TF-IDF vs. Embeddings for RAG

Context: The RAG module needs to search document chunks by semantic similarity.

Chosen: TF-IDF vectorization with cosine similarity.

Rationale: - No external API calls needed — runs entirely locally - Zero cost — no OpenAI embedding API bills - Sufficient accuracy for small document collections (demo scenario) - Simple implementation — scikit-learn's `TfidfVectorizer` is easy to use - Can be upgraded to embeddings later without changing the API contract

Decision 6: Health Check Endpoints

Context: WebBartosz needs to show live service status.

Chosen: Simple GET `/health` endpoints returning `{"status": "ok"}`.

Rationale: - Minimal overhead — no database queries or complex logic - Works with Render's built-in health check monitoring - 5-second timeout to avoid hanging the status display - Consistent format across all 4 services

11. Future Improvements

1. **Task Queue & RAG frontends** — Extract into standalone services with their own URLs, matching the pattern of LinkShort and AI Chat Proxy
2. **CI/CD pipeline** — GitHub Actions for automated testing on push, auto-deploy to Render on merge to main
3. **PostgreSQL migration** — Upgrade from SQLite to PostgreSQL for production readiness (Render offers managed PostgreSQL)
4. **Custom domains** — Point custom domains to Render services (e.g., `chat.bartosz.dev` → AI Chat Proxy)
5. **Authentication across all modules** — Currently only LinkShort and python-portfolio have JWT auth. Add auth to GraphQL Blog and AI Chat Proxy
6. **Rate limiting on unified API** — The python-portfolio API has no rate limiting, making it vulnerable to abuse. Add the same token bucket pattern used in AI Chat Proxy
7. **Usage analytics dashboard** — Build a dashboard showing API usage metrics across all services (requests, errors, latency)
8. **Automated report regeneration** — GitHub Action that regenerates reports when source code changes, keeping documentation in sync
9. **WebSocket auto-reconnect** — The queue tab's WebSocket connection should attempt to reconnect on disconnect with exponential backoff
10. **Animation system** — Consistent animation library across all frontends (currently ad-hoc CSS animations per project)

11. **Error boundary UI** — Each frontend should have proper error states, not just console errors or raw error messages
 12. **Dockerfiles for all services** — Currently only LinkShort has a Dockerfile. Adding Dockerfiles to all services would make local development and deployment more consistent
 13. **Environment variable validation** — Add Pydantic settings validation to all services to catch missing config at startup
 14. **Scheduled health checks** — Use a cron job or Render cron to ping all services every 5 minutes to prevent cold starts
-

12. Appendix: Code Snippets

A.1 AI Chat Proxy — Rate Limiter Dependency

```
async def check_rate_limit(request: Request):
    client_ip = request.client.host
    now = time.time()

    request_counts[client_ip] = [
        req_time for req_time in request_counts[client_ip]
        if now - req_time < RATE_LIMIT_WINDOW
    ]

    if len(request_counts[client_ip]) >= RATE_LIMIT_MAX_REQUESTS:
        raise HTTPException(status_code=429, detail="Rate limit exceeded")

    request_counts[client_ip].append(now)
```

A.2 AI Chat Proxy — SSE Streaming Generator

```
async def stream_generator():
    async for chunk in response.aiter_bytes():
        yield chunk

return StreamingResponse(
    stream_generator(),
    media_type=response.headers.get("content-type", "text/event-stream"),
)
```

A.3 AI Chat Proxy — Frontend SSE Parser

```
const reader = res.body.getReader();
const decoder = new TextDecoder();
let buffer = '';
let content = '';

while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  buffer += decoder.decode(value, { stream: true });
  const lines = buffer.split('\n');
  buffer = lines.pop();
  for (const line of lines) {
    if (line.startsWith('data: ')) {
      const data = line.slice(6).trim();
      if (data === '[DONE]') continue;
      try {
        const parsed = JSON.parse(data);
        const delta = parsed.choices?.[0]?.delta?.content || '';
        content += delta;
        bubble.textContent = content;
      } catch {}
    }
  }
}
```

A.4 python-portfolio — Frontend Serving Catch-All

```
static_dir = Path(__file__).parent.parent / "frontend" / "dist"
if static_dir.exists():
    app.mount("/assets", StaticFiles(directory=str(static_dir / "assets")), name="assets")

@app.api_route("/{full_path:path}", methods=["GET"])
def serve_frontend(full_path: str):
    file = static_dir / full_path
    if file.exists() and file.is_file():
        return FileResponse(str(file))
    return FileResponse(str(static_dir / "index.html"))
```

A.5 python-portfolio — WebSocket Queue Handler (Conceptual)

```
function connectWS(taskId) {
  const ws = new WebSocket(`wss://python-portfolio-y0z8.onrender.com/api/queue/ws/${taskId}`);

  ws.onmessage = (e) => {
    const data = JSON.parse(e.data);
    const bar = document.getElementById(`progress-${taskId}`);
    const status = document.getElementById(`status-${taskId}`);

    bar.style.width = data.progress + '%';
    bar.textContent = data.progress + '%';
    status.textContent = data.status;

    if (data.progress >= 100) {
      status.style.color = 'var(--green)';
      ws.close();
    }
  };

  ws.onerror = () => {
    console.error('WebSocket error for task', taskId);
  };
}
```

A.6 LinkShort — Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

A.7 WebBartosz — Health Check Loop

```
const services = [
  { name: 'shortener', url: 'https://fastapiurl.onrender.com/health' },
  { name: 'blog',      url: 'https://graphql-blog-lxjy.onrender.com/health' },
  { name: 'chat',      url: 'https://ai-chat-proxy-twj4.onrender.com/health' },
  { name: 'portfolio', url: 'https://python-portfolio-y0z8.onrender.com/health' },
];

async function checkAll() {
  const results = await Promise.all(services.map(async (svc) => {
    try {
      const res = await fetch(svc.url, { signal: AbortSignal.timeout(5000) });
      return res.ok;
    } catch { return false; }
  }));

  const online = results.filter(Boolean).length;
  document.getElementById('status').textContent = `${online}/${services.length} online`;
}

checkAll();
setInterval(checkAll, 60000);
```

A.8 Reports Site — Project Card Grid Population

```
const grid = document.getElementById('grid');
projects.forEach((p) => {
  const card = document.createElement('div');
  card.className = 'card';
  card.onclick = () => showReports(p.id);
  card.innerHTML = `
    <div class="icon" style="background:${p.color}22;color:${p.color}">${p.icon}</div>
    <h3>${p.name}</h3>
    <div class="desc">${p.desc}</div>
    <div class="tech">${p.tech.map(t => `<span>${t}</span>`).join('')}</div>
    <a class="btn" onclick="event.stopPropagation();showReports('${p.id}')" href="#">
      View 20 reports &rarr;
    </a>`;
  grid.appendChild(card);
});
```